

OWASP (Open Web Application Security Project) publishes a list of the top risks for mobile applications, helping developers and testers understand critical security issues.

1. Improper Platform Usage

Misuse of platform features or failure to use platform security controls properly (e.g., Android intents, iOS Keychain misuse).

2. Insecure Data Storage

Storing sensitive data insecurely on the device (e.g., in plain text files, SQLite databases without encryption).

3. Insecure Communication

Failing to protect data in transit (e.g., lack of SSL/TLS, using weak protocols).

4. Insecure Authentication

Weak authentication mechanisms, improper session management, or storing credentials insecurely.

5. Insufficient Cryptography

Using weak or flawed cryptographic algorithms or improper implementation of cryptography.

6. Insecure Authorization

Failing to properly restrict user access to resources (e.g., broken access control).

7. Client Code Quality Issues

Bugs in the mobile app code such as buffer overflows, insecure randomness, or improper error handling.

8. Code Tampering

Attackers modify the app binary or resources to change app behavior (e.g., via repackaging or patching).

9. Reverse Engineering

Lack of obfuscation or anti-debugging allows attackers to analyze the app's internals, discover secrets, or modify behavior.

10. Extraneous Functionality

Including hidden backdoors, debug code, or other features not intended for production that can be exploited.

Attacks on Android Apps – Common Vectors

1. Insecure Data Storage

- Android apps sometimes store sensitive data (passwords, tokens) in SharedPreferences, files, or databases without encryption.
- Attackers who gain physical or root access can extract this data.

2. Intent Spoofing and Injection

- Android uses intents to communicate between apps.
- Malicious apps can send crafted intents to exploit vulnerabilities or escalate privileges.

3. Insecure Communication

-

Apps may send data over unencrypted HTTP or weak SSL/TLS configurations, making MITM (Man-in-the-Middle) attacks possible.

4. Reverse Engineering and Code Injection

- Android apps are distributed as APK files, which can be unpacked and analyzed.
- Attackers reverse engineer apps to find hardcoded secrets or inject malicious code.

5. Dynamic Code Loading

- Apps can load code at runtime from external sources, which can be exploited to inject malicious payloads.

6. Insecure Authentication and Session Management

- Flaws in login, token management, or session expiration can lead to unauthorized access.

7. Over-privileged Permissions

- Apps requesting more permissions than necessary increase the attack surface.

8. Code Injection via WebViews

- If WebViews are not secured, they can be exploited with JavaScript injection attacks.